Εργασία 2

Χωρικά Δεδομένα - Διαχείριση Σύνθετων Δεδομένων

ΠΑΥΛΙΔΗΣ ΕΛΕΥΘΕΡΙΟΣ - ΑΜ: 5332 | ΖΩΤΟΣ ΣΩΤΗΡΙΟΣ - ΑΜ: 5227

1. Εισαγωγή

Σκοπός της εργασίας ήταν η υλοποίηση ενός R-tree με τη μέθοδο STR (Sort-Tile-Recursive) καθώς και η υλοποίηση ερωτημάτων αναζήτησης σε χωρικά δεδομένα.

2. Κατασκευή R-tree

Το αρχείο **Rtree.py** αναλαμβάνει την κατασκευή του R-Tree από ένα αρχείο εισόδου με συντεταγμένες (x, y) και την αποθήκευση του σε αρχείο CSV. Χρησιμοποιούμε τη μέθοδο **Sort-Tile-Recursive (STR) Packing** για τη βέλτιστη κατασκευή του δένδρου.

2.1 Παράμετροι χωρητικότητας

Ορίζουμε τις σταθερές για τη μέγιστη και ελάχιστη χωρητικότητα φύλλων και εσωτερικών κόμβων:

```
MAX_LEAF = 51    MIN_LEAF = 20 MAX_INT  = 28    MIN_INT  = 11
```

2.2 Κλάση rTreeNode

Κάθε κόμβος αναπαρίσταται από ένα αντικείμενο **rTreeNode** που φέρει μοναδικό αναγνωριστικό (*id*), σημαία φύλλου (*is_fyllo*), λίστα καταχωρήσεων (*entries*) και το MBR του.

2.3 Υπολογισμός MBR — calculate_mbr()

Η συνάρτηση δέχεται λίστα σημείων ή ορθογωνίων και επιστρέφει το MBR ως *[x_min, y_min, x_max, y_max]*. Η λογική διαφοροποιείται αν η είσοδος αποτελείται από σημεία (2 συντεταγμένες) ή MBR (4 συντεταγμένες).

2.4 Αλγόριθμος STR Packing — pack_nodes()

Για τα **φύλλα**, τα σημεία ταξινομούνται αρχικά κατά τον άξονα X και στη συνέχεια κατά Y εντός κάθε slice, σχηματίζοντας πλήρεις κόμβους έως MAX_LEAF εγγραφών. Για τους **εσωτερικούς κόμβους**, οι εγγραφές ομαδοποιούνται κατά MAX_INT. Η βοηθητική συνάρτηση *update_komvon()* διασφαλίζει ότι ο τελευταίος κόμβος κάθε επιπέδου τηρεί την ελάχιστη πληρότητα.

2.5 Κύρια ροή — main()

Τα σημεία φορτώνονται από αρχείο εισόδου, κατασκευάζονται διαδοχικά τα επίπεδα φύλλων και εσωτερικών κόμβων έως να απομείνει ένας μοναδικός κόμβος (ρίζα), και το αποτέλεσμα το γράφουμε σε αρχείο CSV. Για κάθε επίπεδο εκτυπώνουμε τον αριθμό κόμβων και τη μέση επιφάνεια MBR.

3. Window Range Query — WRQuery.py

Το **Window Range Query** επιστρέφει όλα τα σημεία που βρίσκονται εντός ενός ορθογωνίου παραθύρου *[x_min, y_min, x_max, y_max]*.

3.1 Φόρτωση δένδρου — load_tree()

Η μέθοδος διαβάζει το αρχείο CSV γραμμή προς γραμμή. Αναλύουμε τη κάθε γραμμή με εύρεση ζευγών παρενθέσεων που περιέχουν το αναγνωριστικό και τη γεωγραφική περιοχή. Ο κωδικός του τελευταίου κόμβου αποθηκεύεται ως root_id.

3.2 Συνθήκες επικάλυψης

Χρησιμοποιούνται δύο βοηθητικές μέθοδοι:

- overlaps(bounding_box, window): Ελέγχει αν ένα MBR συμπίπτει με το παράθυρο αναζήτησης χρησιμοποιώντας αποκλεισμό άξονα (axis-aligned separating axis test).
- Is_inside(point, window): Ελέγχει αν ένα σημείο βρίσκεται εντός του παραθύρου με απλό εύρος ελέγχου.

3.3 Αλγόριθμος αναζήτησης — window_range_query()

Με την αναδρομική συνάρτηση εξετάζουμε κάθε κόμβο: αν είναι φύλλο (flag == 0), ελέγχει κάθε σημείο αν βρίσκεται εντός του παραθύρου αλλιώς αν είναι εσωτερικός κόμβος, πάει στα παιδιά των οποίων το MBR επικαλύπτεται με το παράθυρο. Επιστρέφουμε λίστα με τα αναγνωριστικά των εγγραφών που τηρούν το κριτήριο.

4. Distance Range Query — DRQuery.py

Το **Distance Range Query** επιστρέφει όλα τα σημεία που βρίσκονται εντός ακτίνας *r* από ένα κεντρικό σημείο *c*.

4.1 Υπολογισμός αποστάσεων

- point_distance(p1, p2): Υπολογίζει την Ευκλείδεια απόσταση μεταξύ δύο σημείων με τον τύπο sqrt((x1-x2)^2 + (y1-y2)^2).
- min_distance(point, mbr): Υπολογίζει την ελάχιστη δυνατή απόσταση μεταξύ ενός σημείου και ενός MBR. Αν το σημείο βρίσκεται εντός του MBR, η απόσταση είναι 0.

4.2 Αλγόριθμος αναζήτησης — distance_range_query()

Η αναδρομή ακολουθεί την ίδια δομή με το WRQuery: στα φύλλα ελέγχεται η Ευκλείδεια απόσταση κάθε σημείου από το κέντρο, ενώ στους εσωτερικούς κόμβους χρησιμοποιείται

η ελάχιστη απόσταση MBR για **pruning** (αποκλεισμός υπο-δένδρων που δεν μπορούν να περιέχουν αποτελέσματα).

5. k-Nearest Neighbor Query — KNNQuery.py

Το ερώτημα k πλησιέστερων γειτόνων (**kNN Query**) επιστρέφει τα *k* πλησιέστερα σημεία προς ένα σημείο ερωτήματος, χρησιμοποιώντας τον αλγόριθμο **Best-First Search** με ουρά προτεραιότητας (heap).

5.1 Δομή priority queue

Κάθε στοιχείο στο heap αποθηκεύεται ως τριάδα *(distance, type, id)* όπου type = 0 για MBR κόμβους και type = 1 για σημεία φύλλων. Η χρήση του type ως δεύτερο κριτήριο σύγκρισης εξασφαλίζει ότι σε ισοπαλία αποστάσεων τα σημεία εξάγονται πριν από κόμβους.

5.2 Αλγόριθμος — knn_query()

Αρχικά εισάγουμε στο heap τις εγγραφές της ρίζας. Σε κάθε επανάληψη εξάγουμε το στοιχείο με τη μικρότερη απόσταση:

- Αν είναι σημείο (type=1): το προσθέτουμε στα αποτελέσματα — είναι ο επόμενος πλησιέστερος γείτονας.
- Αν είναι κόμβος (type=0): επεκτείνεται — τα παιδιά του (MBR ή σημεία) εισάγονται στο heap με τις αντίστοιχες αποστάσεις τους.

Η διαδικασία συνεχίζεται έως ότου βρεθούν *k* αποτελέσματα ή εξαντληθεί το heap. Ο αλγόριθμος εγγυάται βέλτιστο αποτέλεσμα λόγω της ιδιότητας του Best-First Search.

5.3 Βελτιστοποίηση — pruning

Στο kNN, το pruning γίνεται ουσιαστικά από μόνο του. Αν ένας κόμβος έχει ελάχιστη δυνατή απόσταση (min_distance) μεγαλύτερη από την απόσταση του k-οστού κοντινότερου σημείου που έχουμε ήδη βρει, τότε δεν αξίζει να τον εξετάσουμε. Ο λόγος είναι ότι υπάρχουν ήδη k σημεία πιο κοντά, άρα αυτός ο κόμβος δεν μπορεί να δώσει καλύτερο αποτέλεσμα. Έτσι, δεν θα βγει ποτέ από το heap για επεξεργασία και απλά αγνοείται.